DAY 3

Apache Kafka

Monitoring, Troubleshooting & Diagnostics

Apache Kafka Training Program

Rajesh Pasham

2026

Day 3 Agenda

1

Key Kafka Metrics

Broker Health · Consumer Lag · Controller

2

Monitoring Tools

JMX · Prometheus · Grafana · Burrow

3

Troubleshooting Scenarios

Lag · URPs · Producer & Consumer Issues

4

Broker Log Analysis

Log files · Patterns · Grep techniques

5

Simulating Consumer Lag

Slow consumer · High-volume producer · Recovery

6

Hands-On Labs

7 structured labs · Assessment Quiz

Learning Objectives

- Understand and interpret all critical Kafka JMX and operational metrics
- Analyze Kafka broker and controller logs effectively
- Identify, diagnose, and resolve consumer lag issues confidently
- Simulate and resolve common production incidents
- Troubleshoot Under-Replicated Partitions (URPs) and broker failures
- Apply structured diagnostic workflows for rapid incident resolution

MODULE 1

Key Kafka Metrics

Broker Health · Consumer Lag · Controller Metrics

Rajesh Pasham | Apache Kafka Training

Critical Broker Health Metrics

= 1	= 0	= 0	> 50%
ActiveControllerCount (cluster-wide)	UnderReplicated Partitions (URP)	OfflinePartitions Count	RequestHandler IdleRatio

Alert Thresholds & Actions

ActiveControllerCount	≠ 1	Check ZooKeeper/KRaft, restart controller broker
UnderReplicatedPartitions	> 0 for > 30s	Check broker health, disk space, network
OfflinePartitionsCount	> 0 (ANY)	IMMEDIATE — producers/consumers are blocked
RequestHandlerIdleRatio	< 0.20 (20%)	Broker overloaded — scale out or reduce load

Understanding Consumer Lag

Consumer Lag = Log End Offset (LEO) − Current Committed Offset

Log End Offset (LEO)

The offset of the next message to be written. This is the "latest" position in the partition — what the producer has written.

Current Offset

The last offset committed by the consumer group. This is how far the consumer has actually processed messages.

Consumer Lag

LEO minus Current Offset. Zero = consumer is caught up. Growing lag = consumer is falling behind the producer.

Checking Consumer Lag — CLI

COMMAND

```
kafka-consumer-groups.sh \
  --bootstrap-server localhost:9092 \
  --describe --group my-consumer-group
```

CURRENT-OFFSET	Last offset committed — stops growing = stuck consumer
LOG-END-OFFSET	Latest produced offset — if growing fast vs. CURRENT = growing lag
LAG	LEO − Current. Key: Is it STABLE or GROWING?
CONSUMER-ID	Shows '-' if no active consumer is assigned to partition

Kafka Controller Health Metrics

The Controller is the brain of the cluster — instability here affects the entire cluster

`ActiveControllerCount`	= 1	0 or > 1	Exactly ONE broker must be the active controller at all times
`LeaderElectionRateAndTimeMs`	≈ 0	Frequent	Non-zero rate means partitions are changing leaders
`UncleanLeaderElectionsPerSec`	= 0	ANY > 0	Non-ISR replica elected = DATA LOSS. Must always be zero.
`OfflinePartitionsCount`	= 0	ANY > 0	No leader available — producers & consumers blocked

Any unclean leader election means data loss has occurred — investigate immediately!

MODULE 2

Monitoring Tools

JMX · Prometheus · Grafana · Burrow

Production Monitoring Stack

Kafka Broker	JMX Exporter	Prometheus	Grafana
JMX port: 9999 KAFKA_JMX_OPTS	Java agent Port 7071 /metrics endpoint	Scrapes metrics every 15s Time-series DB	Dashboard ID 7589, 11962 Visualizations

+ Burrow (LinkedIn) — Dedicated consumer lag monitoring with trend-based health states: OK → WARNING → ERROR → STALL → STOP

Burrow Consumer Health States

Burrow evaluates lag TRENDS — not just absolute values — to determine consumer health

OK	Consumer is making progress. Lag is stable or decreasing. No action needed.
WARNING	Consumer lag is increasing but consumer is still active and committing.
ERROR	Consumer is stuck — not committing offsets. Restart consumer, investigate.
STALL	Consumer is committing but not consuming new messages. Check consumer logic.
STOP	Consumer has stopped entirely. Immediate intervention required.

```
curl http://burrow-host:8000/v3/kafka/local/consumer/<group>/status
```

MODULE 3

Troubleshooting

Consumer Lag · URPs · Producer · Consumer Issues

Consumer Lag — Diagnostic Workflow

1 **Quantify the lag**

```
kafka-consumer-groups.sh --bootstrap-server broker:9092 --describe --group <group>
```

2 **All partitions or specific?**

Check LAG column — all partitions lagging vs. only specific ones

3 **Check consumer throughput**

JMX: `kafka.consumer:type=consumer-fetch-manager-metrics,name=records-consumed-rate`

4 **Check producer rate**

`kafka.server:type=BrokerTopicMetrics,name=MessagesInPerSec` — is input spiking?

5 **Review consumer JVM & GC logs**

Look for long GC pauses correlating with lag spikes

Consumer Lag — Root Causes & Fixes

Consumer too slow

Optimize consumer logic; reduce processing time per message

Too few consumer instances

Add more consumers (up to partition count)

Consumer GC pauses

Tune JVM GC settings; reduce heap pressure

Rebalancing loop

Increase max.poll.interval.ms; fix consumer stability

Network bottleneck

Move consumer closer to Kafka; check NIC throughput

Downstream system slow

Add buffering; async writes to downstream; circuit breaker

Under-Replicated Partitions (URPs)

What is a URP?

A partition where one or more replicas have fallen OUT of the ISR (In-Sync Replica set).

This reduces fault tolerance — if the leader fails while URP > 0, you risk data loss or an outage.

URP = 0 is the only acceptable production value.

Check command:

```
kafka-topics.sh --bootstrap-server broker:9092 --describe --under-replicated-partitions
```

Common Causes & Fixes

Broker is down

Restart broker; check disk/OOM

Follower too slow

Increase replica.fetch.max.bytes; check disk I/O

Network partition

Check network routes; firewall rules between brokers

Disk full

Free disk space IMMEDIATELY — broker will stop!

Error Signatures Quick Reference

`TOPIC_AUTHORIZATION_FAILED`	No Read/Write ACL on topic	Topic:<name> Read or Write ACL
`UNKNOWN_TOPIC_OR_PARTITION`	Topic missing OR no Describe ACL	Create topic or add Describe ACL
`GROUP_AUTHORIZATION_FAILED`	No Read ACL on consumer group	Group:<group-id> Read ACL
`TimeoutException (producer)`	Broker unreachable or buffer full	Check network; increase buffer.memory
`RecordTooLargeException`	Message exceeds max.request.size	Increase message.max.bytes on broker+producer
`RebalanceInProgressException`	Consumer group is actively rebalancing	Increase session.timeout.ms; fix consumer health
`NotLeaderForPartitionException`	Stale metadata — partition leader moved	Auto-resolves; check metadata.max.age.ms

MODULE 4

Broker Log Analysis

Log Files · Key Patterns · Grep Techniques

Kafka Log Files — What Lives Where

`server.log`	ALWAYS	Main broker operational log — ALL key events. First stop in any investigation.
`controller.log`	HIGH	Controller-specific events — leader elections, topic changes, broker failures.
`state-change.log`	HIGH	Leader/follower state transitions per partition. Use for ISR and URP investigations.
`kafka-authorizer.log`	MEDIUM	ACL authorization decisions — DENIED entries show exactly what ACL is missing.
`kafka-gc.log`	MEDIUM	JVM garbage collection. Correlate long GC pauses with request latency spikes.
`log-cleaner.log`	LOW	Log compaction operations. Check when compacted topics have issues.

Critical Log Patterns to Know

HIGH	`Shrinking ISR from X to Y`	A replica fell behind and was removed from ISR — investigate the lagging follower
INFO	`Expanding ISR from X to Y`	A replica caught up and rejoined ISR — recovery in progress
CRITICAL	`Broker N has been marked dead`	Broker failure detected by controller — immediate investigation needed
CRITICAL	`No space left on device`	Disk full — broker will stop writing. Free space IMMEDIATELY
HIGH	`Error in fetch from leader`	Follower cannot fetch from leader — check network and leader health
CRITICAL	`CorruptRecordException`	Data corruption detected in log segment — potential data loss

Essential Log Grep Commands

```
# ISR changes
grep -i 'shrinking isr\|expanding isr' /var/log/kafka/server.log
```

```
# Broker failures
grep 'marked dead\|shut down\|shutdown' /var/log/kafka/server.log
```

```
# All ERRORs
grep 'ERROR' /var/log/kafka/server.log | tail -50
```

```
# Disk issues
grep -i 'IOException\|No space left\|disk' /var/log/kafka/server.log
```

```
# Controller events
grep 'becomes active controller\|Elect leader' /var/log/kafka/controller.log
```

```
# GC impact
grep -i 'GC overhead\|OutOfMemoryError' /var/log/kafka/server.log
```

MODULE 5

Simulating Consumer Lag

Slow Consumer · High-Volume Producer · Recovery Strategies

3 Ways to Simulate Consumer Lag

METHOD 01

Slow Consumer

Introduce artificial delay in consumer processing (e.g. time.sleep(0.05)) so throughput is lower than producer rate.

```
time.sleep(0.05)  # 50ms = max 20 msgs/sec
```

METHOD 02

High-Volume Producer Flood

Use kafka-producer-perf-test.sh to flood the topic faster than the consumer can process.

```
kafka-producer-perf-test.sh --num-records 100000 --throughput -1 ...
```

METHOD 03

Stop Consumer / Keep Producing

Kill all consumer instances while production continues. Lag grows continuously until consumer is restarted.

```
watch -n 2 'kafka-consumer-groups.sh ... --describe --group test-group'
```

Consumer Lag Recovery Strategies

1. Reset Offset to Latest

DATA LOSS

Skip all lagged messages — consumer jumps to the current end of the topic. Fast but loses messages. Use only for non-critical data streams.

2. Scale Up Consumers

SAFE

Add more consumer instances (up to partition count). Each instance handles fewer partitions, increasing total throughput. Best approach.

3. Optimize Consumer Code

SAFE

Reduce processing time per message. Use batch DB writes, async I/O, optimize transformations. Addresses root cause directly.

4. Pause Non-Critical Consumers

TRADE-OFF

Temporarily pause lower-priority consumers to reduce broker load, giving critical consumers more resources to catch up.

HANDS-ON

7 Lab Exercises

Structured · Scenario-Based · Progressive Difficulty

Lab Overview

1	Kafka Metrics Exploration with JMX & CLI	30 min	Beginner
2	Setting Up Consumer Lag Monitoring	30 min	Intermediate
3	Simulating and Resolving Consumer Lag	45 min	Intermediate
4	Inducing & Diagnosing Under-Replicated Partitions	45 min	Advanced
5	Kafka Broker Log Analysis	30 min	Intermediate
6	Producer Troubleshooting Scenarios	30 min	Intermediate
7	End-to-End Monitoring Dashboard Challenge	30 min	Advanced

Total Lab Time: ~4 hours | Prerequisites: Kafka on localhost:9092 | Python 3 + confluent-kafka

Lab 3: Simulate Consumer Lag with Python

slow_consumer.py

```
from confluent_kafka import Consumer
import time

conf = {
  'bootstrap.servers': 'localhost:9092',
  'group.id': 'slow-consumer-group',
  'auto.offset.reset': 'earliest'
}
c = Consumer(conf)
c.subscribe(['kafka-lab-topic'])
while True:
    msg = c.poll(1.0)
    if msg:
        time.sleep(0.05)  # 50ms delay
        # lag grows if rate > 20/sec
```

Lab Steps

1. Reset consumer offset to earliest
2. Start slow_consumer.py (Terminal 1)
3. Flood topic with 100K messages (Terminal 2)
4. Watch lag grow with 'watch' command (Terminal 3)
5. Record lag every 30s for 2 minutes
6. Apply fix: reset to latest OR optimize speed
7. Verify lag drops to zero

Lab 4: Induce & Diagnose URPs

Requires a 3-broker cluster. Demonstrates real ISR shrink/expand lifecycle.

SETUP

1. Create replicated-topic with RF=3
2. Verify all ISRs healthy (Replicas = Isr)
3. Confirm URP count = 0

BREAK

1. Stop one broker (e.g. port 9094)
2. Immediately check --under-replicated-partitions
3. Record which partitions lost ISR member

DIAGNOSE

1. Check broker-api-versions.sh for dead broker
2. Grep server.log for 'marked dead' entry
3. Check ReplicaManager MaxLag via JMX

RECOVER

1. Restart the stopped broker
2. Watch ISR recovery with 'watch' command
3. Record recovery time duration

Day 3 Assessment Quiz

20 Questions | Passing Score: 75% (15/20) | Time: 20 minutes

Q1	What should ActiveControllerCount equal across an entire Kafka cluster?	b) Exactly 1
Q2	Consumer Lag is calculated as:	c) Log End Offset – Current Offset
Q3	Which log file should you check FIRST in any Kafka investigation?	a) server.log
Q4	An unclean leader election means:	b) Non-ISR replica elected — potential DATA LOSS
Q5	What config controls when a follower is removed from ISR?	c) replica.lag.time.max.ms

Day 3 Key Takeaways

Metrics That Matter

ActiveControllerCount=1, URP=0, OfflinePartitions=0 are non-negotiable. Monitor RequestHandlerIdleRatio for load.

Lag Is a Trend

Stable lag at 50K is fine. Growing lag is a crisis. Always observe DIRECTION, not just the absolute value.

Logs Tell the Full Story

server.log first, always. ISR shrinks, broker failures, disk errors — all leave clear signatures.

Structured Diagnosis

Use the 5-step workflow: Quantify → Isolate → Measure → Root Cause → Fix. Don't jump to solutions.

Day 3 Complete!

Monitoring · Troubleshooting · Diagnostics

Rajesh Pasham

Apache Kafka Training Program | 2026